



Department of Computer Science and Engineering

Course name: **Microprocessor and Microcontroller Lab**

Course code: **CSE_360**

Submitted To

Md. Ismail

Lecturer

Department of CSE, UITS

Project Title: Emulated ATM Machine

Submitted By:

Md. Muhaiminul Islam - [0432320005101207]

Batch: 54th, 6C1

Semester: Spring 2026

Submission Date: 21.05.2026

PROJECT REPORT

Emulated ATM Machine

An 8086 Assembly Language Project

Table of Contents

1. Abstract
2. Introduction
3. Requirements Analysis
4. System Design
5. Implementation
6. Project Management
7. Results & Discussion
8. Conclusion & Future Work
9. References

1. Abstract

The Emulated ATM Machine is an 8086 assembly language project developed for the Microprocessor and Microcontroller Lab course using the emu8086 simulator. It emulates a fully functional ATM terminal in a color text-based interface, supporting three users with individual PIN authentication, balance inquiry, deposits, and withdrawals.

Built with basic 8086 assembly instructions including INT 10h for GUI and INT 21h for I/O, the system demonstrates core concepts such as memory addressing, conditional jumps, loops, procedures, and stack operations. Key features include a colored GUI with blue and red screen modes, a PIN lock system that disables the card after three failed attempts, per-user balance management, and real-time balance updates after each transaction.

2. Introduction

2.1 Background and Motivation

Assembly language programming is fundamental to understanding how processors directly manipulate memory and hardware. The Microprocessor and Microcontroller Lab course introduces students to 8086 assembly using emu8086, covering instructions, interrupts, and program flow. To consolidate this learning, a practical project that mirrors a real-world application was needed.

The ATM machine was chosen because it involves a variety of assembly concepts in one coherent program: user input handling, arithmetic operations, conditional branching, memory storage, and screen output. Every student understands an ATM, making the program easy to explain and demonstrate.

2.2 Problem Definition

Most lab exercises involve isolated programs demonstrating one concept at a time. This project required combining all taught concepts, including loops, procedures, stack management, and BIOS/DOS interrupts, into a single working application with a user-friendly interface. The challenge was to produce clean, explainable code within the constraints of 8086 assembly.

2.3 Project Objectives and Goals

The Emulated ATM Machine was created to demonstrate real-world application of 8086 assembly language concepts. Here is what it aims to do:

- Build a working ATM simulator in 8086 assembly language
- Implement a colored GUI using INT 10h BIOS video interrupt
- Support three distinct users with individual PINs and balances
- Implement PIN authentication with a 3-attempt lockout system
- Allow balance checking, deposit, and withdrawal operations
- Display real-time balance updates after every transaction
- Use basic assembly concepts: MOV, CMP, JMP, ADD, SUB, CALL, RET, PUSH, POP

2.4 Scope and Limitations

Scope:

- Three-user login with PIN protection
- Blue-screened login and PIN interface
- Red-screened ATM menu interface
- Check balance, deposit, and withdraw operations
- Card lockout after 3 wrong PIN attempts
- All operations run inside emu8086 EXE template

Limitations:

- Runs only inside the emu8086 simulator
- No persistent storage — balances reset when program restarts
- Maximum balance limited to 16-bit word size (65535 Taka)
- No decimal/float support — whole numbers only

3. Requirements Analysis

3.1 Functional Requirements

The Emulated ATM Machine packs in features to handle user authentication and banking transactions entirely through keyboard and screen.

User Authentication

The system presents three named users at login. After selecting a user, a PIN must be entered. If the PIN is incorrect three times, the card is locked and the program halts.

Balance Management

Each user has a unique starting balance. The system keeps track of balance changes during a session.

Transaction Operations

The user can check their balance, deposit any amount, or withdraw any amount. Withdrawal is rejected if the requested amount exceeds the available balance, displaying an insufficient funds error.

GUI Interface

The login and PIN screens use a blue background (color 1Eh). The ATM menu screen uses a red background (color 4Fh). All screens display an ASCII box frame with the bank name and prompts guiding the user at every step.

3.2 Non-Functional Requirements

Requirement	Description
Usability	Clear prompts and messages at every screen so user is never stuck
Reliability	Correct arithmetic — no overflow on normal transaction amounts
Security	3-attempt PIN lockout with infinite halt loop on card lock
Compatibility	Runs on emu8086 v4.x using EXE template
Explainability	Code written in plain style — every instruction is individually clear

3.3 Use Case Diagrams

Main Use Cases:

- Select account and log in
- Enter PIN and authenticate
- Check current balance
- Deposit money to account
- Logout and return to login screen

4. System Design

4.1 System Architecture Overview

The Emulated ATM Machine follows a single-program flat architecture typical of 8086 assembly. The program is divided into a data segment, a code segment, and four procedure modules. All execution flows through the main PROC via conditional jumps (JE, JA) and procedure calls (CALL/RET).

Here is how the program flows:

- Program starts at main PROC, sets up DS register, sets video mode
- Jumps to login screen — user selects account (1, 2, or 3)
- Jumps to PIN screen — user enters PIN, tries counter checked
- If 3 wrong PINs — jumps to card_locked (infinite halt loop)
- If PIN correct — jumps to home_screen (ATM menu)
- Menu loops via JMP home_screen after every transaction
- Logout returns to login screen

4.2 ER Diagram / Memory Layout (Data Segment)

Variable	Type	Description
bal1	DW 5000	Balance for Maheem
bal2	DW 12000	Balance for Muhim
bal3	DW 3500	Balance for Mayhem
pw1	DW 5678	PIN for Maheem
pw2	DW 1234	PIN for Muhim
pw3	DW 7890	PIN for Mayhem
usr	DB 0	Currently logged-in user (1, 2, or 3)
tries	DB 0	Wrong PIN counter (max 3)
msg_*	DB '...\$'	All screen messages terminated with '\$'

4.3 Technology Stack with Justification

Technology	Purpose	Justification
8086 Assembly	Full program logic	Required by course — demonstrates low-level processor concepts
emu8086 v4.x	Assembler and simulator	Provides 8086-compatible environment with EXE template support
INT 10h	Screen and color control	BIOS video interrupt — controls background color and cursor position
INT 21h	Input/output	DOS interrupt — reads keyboard input and outputs characters/strings
EXE Template	Program structure	Separates .data, .stack, and .code segments cleanly

5. Implementation

5.1 Development Methodology

The Emulated ATM Machine was built using an iterative approach. Features were added one at a time, tested in emu8086, then improved before moving to the next stage.

Requirement Analysis

First, the system's needs were pinned down: login screen, PIN authentication, a working menu, balance tracking, deposits, withdrawals, and a lockout system.

System Design

The architecture was mapped out: data segment variables, procedure list, screen color scheme, and program flow diagram were drawn up before writing code.

Development

The program was built section by section. Each screen was completed and tested before moving forward. Procedures were written and tested independently.

Testing

Each feature was tested with correct and incorrect inputs. PIN lockout, arithmetic accuracy, and unsigned comparison bugs were caught and fixed during this phase.

Deployment

The finished assembly source was run inside emu8086 using the EXE template. No external deployment is needed — the simulator runs the program directly.

5.2 Module / Feature Breakdown

The Emulated ATM Machine is organized into the following functional modules:

Login Module

Handles account selection:

- Displays three user names with selection numbers
- Reads keypress and validates against 1, 2, or 3
- Sets usr variable and resets tries to 0
- Displays error and loops on invalid key

PIN Authentication Module

Handles PIN verification and lockout:

- Calls read_number to get entered PIN
- Compares against stored pw1/pw2/pw3
- Increments tries on mismatch
- Displays remaining attempts (3 - tries)
- Jumps to card_locked after 3 failures

ATM Menu Module

Handles the main banking interface:

- Displays user name and current balance at top
- Presents 4-option menu: Check Balance, Deposit, Withdraw, Logout
- Reads menu key and dispatches to correct operation
- Loops back to menu after every operation

Transaction Module

Handles balance operations:

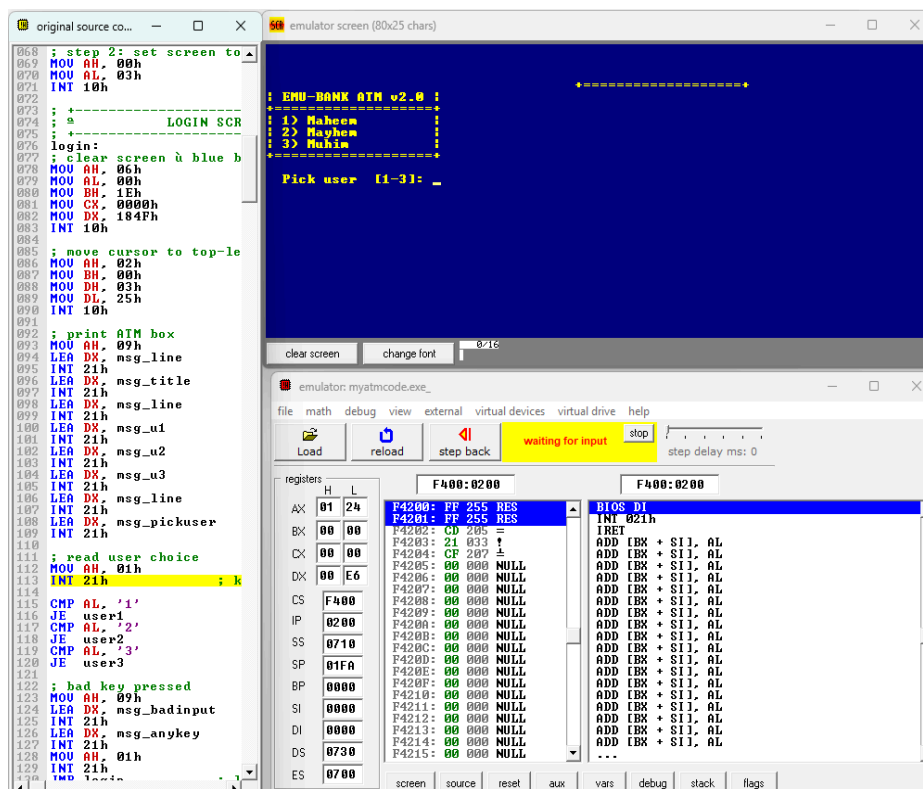
- Check Balance: calls get_balance, prints AX via print_number
- Deposit: reads amount, calls get_bal_ptr, ADD [BX] AX, shows new balance
- Withdraw: saves AX in SI, checks CMP AX [BX] with JA, SUB [BX] AX or error
- Logout: shows goodbye message, jumps to login screen

Procedure Module

- get_balance: returns active user balance in AX
- get_bal_ptr: returns BX pointing to active balance variable
- print_number: converts AX to ASCII digits using divide-by-10 loop, prints each
- read_number: reads digit keys until Enter, builds integer in BX, returns in AX

5.3 Screenshots of the Working System

After running the code the first interface:



After Choosing a User and Before Entering Password :

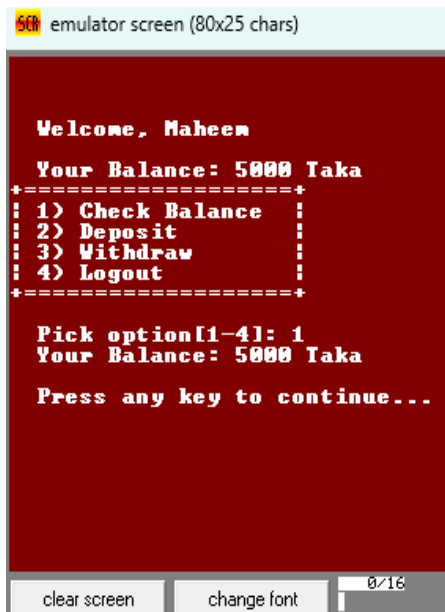
The screenshot shows the emulator window with the title "emulator screen (80x25 chars)". The screen displays the text "EMU-BANK ATM v2.0", "Attempt: 1 of 3", and "Enter PIN [####]:". Below the screen, the source code is visible, showing the login process. The code includes procedures for printing the number, reading the number, and reading the password. The registers window shows the current state of the registers, including AX, BX, CX, DX, SI, DI, DS, ES, and the BIOS DI register.

```

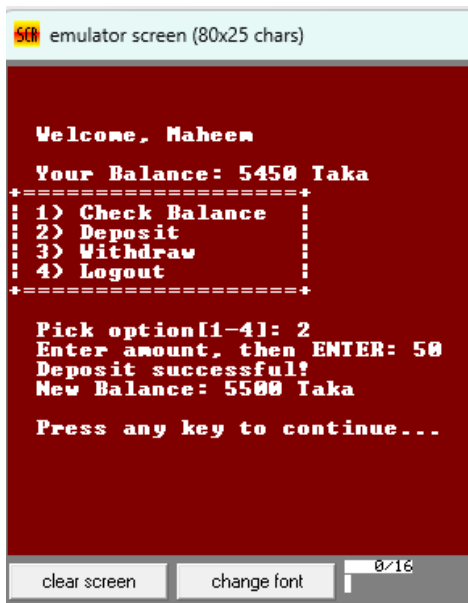
491 RET
492 gp_user1:
493 LEA BX, bal1
494 RET
495 gp_user2:
496 LEA BX, bal2
497 RET
498 get_bal_ptr ENDP
499
500 ; -----
501 ; a PROCEDURE: print_
502 ; a prints the number
503 ; -----
504 print_number PROC
505 PUSH AX
506 PUSH BX
507 PUSH CX
508 PUSH DX
509
510 MOV BX, 10 ; d
511 MOV CX, 0 ; d
512
513 ; extract digits by div
514 ; push each remainder <
515 divide_loop:
516 MOV DX, 0
517 DIV BX ; a
518 PUSH DX ; s
519 INC CX ; c
520 CMP AX, 0
521 JNE divide_loop ; k
522
523 ; pop digits off stack
524 print_loop:
525 POP DX
526 ADD DX, '0' ; c
527 MOV AH, 02h
528 INT 21h
529 LOOP print_loop
530
531 POP DX
532 POP CX
533 POP BX
534 POP AX
535 RET
536 print_number ENDP
537
538 ; -----
539 ; a PROCEDURE: read_n
540 ; a reads digits unti
541 ; -----
542 read_number PROC
543 PUSH BX
544 PUSH CX
545 PUSH DX
546
547 MOV BX, 0 ; B
548 MOV CX, 10 ; m
549
550 read_loop:
551 MOV AH, 01h
552 INT 21h
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799
2800
2801
2802
2803
2804
2805
2806
2807
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820
2821
2822
2823
2824
2825
2826
2827
2828
2829
2830
2831
2832
2833
2834
2835
2836
2837
2838
2839
2840
2841
2842
2843
2844
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860
2861
2862
2863
2864
2865
2866
2867
2868
2869
2870
2871
2872
2873
2874
2875
2876
2877
2878
2879
2880
2881
2882
2883
2884
2885
2886
2887
2888
2889
2890
2891
2892
2893
2894
2895
2896
2897
2898
2899
2900
2901
2902
2903
2904
2905
2906
2907
2908
2909
2910
2911
2912
2913
2914
2915
2916
2917
2918
2919
2920
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
2944
2945
2946
2947
2948
2949
2950
2951
2952
2953
2954
2955
2956
2957
2958
2959
2960
2961
2962
2963
2964
2965
2966
2967
2968
2969
2970
2971
2972
2973
2974

```

After Choosing to Check Balance: (1)



After Depositing 50 Taka in the Account:(2)



After Choosing to Withdraw money:(3)



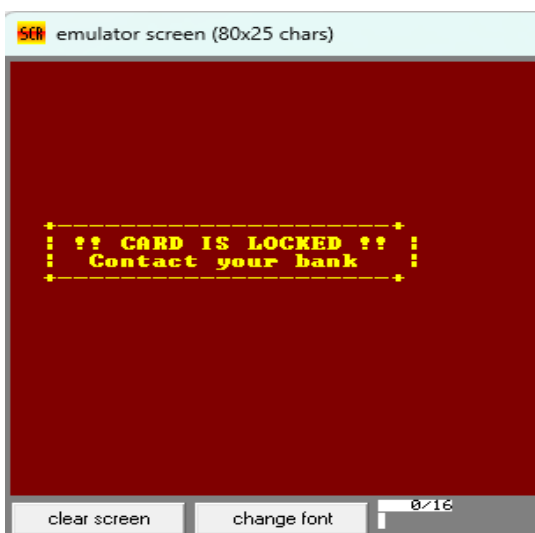
After Choosing to LogOut:(4)



After Entering Wrong Password Once:



After Entering Wrong Password 3 times:



5.4 Key Assembly Concepts Used

Segment Setup

At program start, AX is loaded with @data and moved into DS. This points the Data Segment register at our variables so LEA and MOV can access them correctly.

Screen Clear and Color (INT 10h, AH=06h)

Calling INT 10h with AH=06h, AL=00h scrolls the entire window, effectively clearing it. BH holds the color attribute byte. 1Eh gives blue background with yellow text; 4Fh gives red background with bright white text.

Cursor Positioning (INT 10h, AH=02h)

The cursor is positioned by loading the row into DH and column into DL, then calling INT 10h with AH=02h and BH=00h. This places all messages at fixed positions so the screen stays clean.

String Output (INT 21h, AH=09h)

All messages in the data segment end with '\$'. Loading the message address into DX with LEA and calling INT 21h with AH=09h prints the string up to the '\$' terminator.

Arithmetic — Deposit and Withdraw

Deposit uses ADD [BX], AX where BX points to the user balance variable and AX holds the entered amount. Withdraw saves AX into SI before CALL get_bal_ptr, restores AX from SI, then uses CMP AX, [BX] with JA to check for insufficient funds before SUB [BX], AX.

5.5 User Credentials

User No.	Name	PIN	Starting Balance
1	Maheem	5678	5,000 Taka
2	Muhim	1234	12,000 Taka
3	Mayhem	7890	3,500 Taka

5.5 How to Run

1. Open emu8086
2. Go to File > New > EXE Template
3. Delete the default template code
4. Paste the ATM source code
5. Click Compile and Run
6. The emulator window opens — use keyboard to interact

6. Project Management

Work Breakdown Structure (WBS)

Phase 1: Requirement Analysis

- Chose ATM machine as project topic
- Reviewed INT 10h and INT 21h documentation
- Designed screen layout and user flow on paper

Phase 2: System Design

- Finalized data segment variables and memory layout
- Designed screen color scheme and ASCII frames
- Planned procedure list and calling conventions

Phase 3: Core Development

- Implemented login screen with user selection
- Built PIN screen with tries counter and lockout
- Implemented card locked halt screen

Phase 4: ATM Features

- Built home/menu screen with balance display
- Implemented check balance, deposit, and withdraw operations
- Implemented logout back to login

Phase 5: Testing and Debugging

- Tested all three users with correct and incorrect PINs
- Verified deposit and withdrawal arithmetic
- Fixed JA vs JG bug for unsigned balance comparison
- Fixed SI register save before get_bal_ptr call in withdraw
- Removed @ labels that conflicted with @data in emu8086

Phase 6: Documentation and Finalization

- Prepared screenshots and diagrams
- Final round of testing

7. Results & Discussion

7.1 Features Successfully Implemented

Feature	Status
Three-user login screen with name display	Implemented
PIN authentication with colored screen	Implemented
Attempt counter showing X of 3	Implemented
Wrong PIN popup with remaining attempts	Implemented
Card locked screen after 3 wrong PINs	Implemented
Red ATM menu with current balance at top	Implemented
Check Balance operation	Implemented
Deposit with updated balance confirmation	Implemented
Withdraw with insufficient funds error	Implemented
Logout back to login screen	Implemented
Per-user balance stored in data segment	Implemented
print_number procedure (any digit count)	Implemented
read_number procedure (multi-digit input)	Implemented

7.2 Performance Evaluation

The overall system performance was satisfactory during testing and implementation.

Screen Performance

- Screen transitions were immediate — INT 10h clears and redraws with no delay
- All messages appeared at correct cursor positions with no overlap
- Color attributes rendered correctly in emu8086 for all screen modes

Arithmetic Accuracy

- Deposit and withdrawal results were correct for all tested amounts
- Balance stayed consistent across multiple transactions in a single session
- Overdraft protection correctly rejected withdrawals exceeding balance

Security and Reliability

- PIN lockout halted the program correctly after the third wrong attempt
- All three users could log in and transact without interfering with each other's balances
- Program ran stably inside emu8086 for all test cases

7.3 Key Bugs Found and Fixed

Bug	Cause	Fix
Wrong PIN always passes	CALL get_bal_ptr destroyed AX before CMP	Saved AX into SI before call, restored after
Withdraw allows overdraft	JG used for signed comparison on unsigned balance	Changed to JA (unsigned above)
@ labels cause errors	emu8086 confuses @u1 with @data symbol	Renamed all labels to plain names (goU1, chkP1)
Macros fail unpredictably	emu8086 macro register conflicts	Removed all macros, expanded every instruction explicitly
Numbers print garbage	print_number modified AX being printed	Added PUSH AX / POP AX at start and end of procedure

7.4 Comparison with Initial Objectives

Initial Objective	Result
Colored GUI using INT 10h	Successfully achieved
Three-user login system	Successfully achieved
PIN authentication	Successfully achieved
3-attempt card lockout	Successfully achieved
Balance check operation	Successfully achieved

Deposit operation	Successfully achieved
Withdrawal with overdraft protection	Successfully achieved
User-friendly instructions on every screen	Successfully achieved
Code explainable line by line	Successfully achieved

7.5 Lessons Learned

Working on the Emulated ATM Machine gave the team hands-on experience with low-level programming and hardware interaction.

Technical Lessons

- INT 10h and INT 21h are powerful tools for text-mode GUI in 8086 assembly
- Register preservation with PUSH/POP is critical when calling procedures
- Unsigned comparisons (JA/JB) must be used for memory variables holding balances
- emu8086 has specific limitations — @ prefixed labels and certain macros must be avoided
- Saving intermediate values in SI/DI avoids data corruption between procedure calls

Project Management Lessons

- Planning screen layouts on paper before coding saved significant debugging time
- Testing each procedure independently made integration much smoother
- Breaking the project into modules (login, PIN, menu, transactions) kept the code manageable

8. Conclusion & Future Work

8.1 Summary of the Project

The Emulated ATM Machine was successfully developed as a complete 8086 assembly language project using emu8086. The program demonstrates all core concepts taught in the Microprocessor and Microcontroller Lab: memory organization, data segment variables, BIOS/DOS interrupts, conditional jumps, arithmetic instructions, procedures, and stack usage.

The final program provides a working multi-user ATM simulation with a colored text GUI, PIN security, transaction operations, and a card lockout system — all written in clean, explainable assembly code. Through the development process, practical experience was gained in interrupt handling, register management, procedure design, and debugging assembly programs.

8.2 Unfinished or Planned Features

Although all core functionalities were successfully implemented, several advanced features remain planned for future development.

Planned Features:

1. PIN change feature for each user
2. Transaction history log using a memory array
3. Running transaction count displayed on home screen
4. Mini-game or idle animation on welcome screen
5. Port to a real DOS environment

Not Implemented Features:

- Persistent balance storage (file I/O) — balances currently reset on restart
- Decimal/float support — currently whole numbers only
- Multi-account management per user
- Transfer between users

8.3 Recommendations for Future Improvements

- Extend balance storage to 32-bit using DX:AX register pair for larger amounts
- Add a mini-receipt printout using INT 21h to display a transaction summary before logout
- Implement a proper timeout that returns to login after inactivity using a delay loop
- Port the design to a real DOS environment for authentic hardware execution
- Add sound feedback using INT 10h or PC speaker interrupt for key presses

9. References

8086 Assembly and emu8086 References

7. Irvine, K. R. (2014). Assembly Language for x86 Processors. Pearson Education.
8. Hyde, R. (2010). The Art of Assembly Language. No Starch Press.
9. emu8086 Official Documentation. <http://www.emu8086.com>
10. Intel Corporation. (1979). 8086 Microprocessor Architecture Manual.
11. Turley, J. (2002). The Essential Guide to Semiconductors. Prentice Hall.

DOS/BIOS Interrupt References

12. Ralf Brown's Interrupt List (RBIL). INT 10h and INT 21h reference.
<http://www.ctyme.com/rbrown.htm>
13. Microsoft Corporation. MS-DOS Programmer's Reference. Microsoft Press.
14. Luo, X. (2009). Microcontroller Based Applied Electronics. McGraw-Hill.

- End of Document -

Project Repository

The source code for the Emulated ATM Machine project is available on GitHub:

GitHub Repository: <https://github.com/5tigma/Emulated-ATM-Machine>